Lecture Notes: Hyperparameter Search Strategies

CIC1205 - Machine Learning

Eduardo Bezerra

March 16, 2026

These notes accompany the *Model Selection* notes. Read them **before class**, at your own pace. By the end of your reading, you should be able to answer the following questions:

1. What is a hyperparameter search strategy, and how does it differ from an evaluation method (holdout, CV, nested CV)?
2. What is the computational cost of grid search as a function of the number of hyperparameters? Why does this cost grow exponentially?
3. Why does random search outperform grid search when some hyperparameters have little influence on model performance?
4. What is a surrogate model in Bayesian optimization, and what is the role of the acquisition function?
5. In which practical situation would you choose each of the three strategies: grid search, random search, or Bayesian optimization?

Estimated reading time: 60–80 minutes.
Prerequisite: *Model Selection* notes (especially the sections on k-fold CV and nested CV).

Contents

1 Introduction **2**
1.1 Motivation: Why Hyperparameter Search Is Hard . . . 2
1.2 Two Complementary Components . . . 2

2 Grid Search **3**
2.1 Procedure . . . 3
2.2 Worked Example: Grid Search with 5-Fold CV . . . 4
2.3 Advantages . . . 6
2.4 Limitations . . . 6
2.5 When to Use . . . 6

3 Random Search **7**
3.1 Procedure . . . 7
3.2 Worked Example: Marginal Coverage in Practice . . . 7
3.3 Advantages . . . 11
3.4 Limitations . . . 11
3.5 When to Use . . . 11

4 Bayesian Optimization **11**
4.1 Core Idea . . . 12
4.2 Advantages . . . 12
4.3 Limitations . . . 13
4.4 When to Use . . . 14

1 Introduction

In the *Model Selection* notes you learned how to *evaluate* a model configuration reliably: holdout, k-fold CV, and nested CV. In those examples, the choice of *which* configurations to test was treated implicitly: in the section on k-fold CV, different values of C and γ for an SVM were evaluated; in the nested CV example, three values of `max_depth` were tested. These lectures notes make that process explicit: **what rule determines which configurations enter the evaluation loop?** That is precisely the role of a hyperparameter search strategy.

1.1 Motivation: Why Hyperparameter Search Is Hard

To understand why search strategy matters, consider a concrete scenario.

Scenario. A practitioner is building a tumor-classification model. She is using a gradient-boosted tree and must tune three hyperparameters: learning rate $\eta \in [10^{-4}, 10^{-1}]$, tree depth $d \in \{2, 4, 6, 8, 10\}$, and regularization strength $\lambda \in [10^{-3}, 10^{0}]$. She decides to evaluate 5 values per hyperparameter. Each model training takes approximately 2 hours on her machine. With a grid of $5 \times 5 \times 5 = 125$ configurations and 5-fold cross-validation, the total wall-clock time is:

$$125 \times 5 \times 2 \text{ h} = \mathbf{1{,}250} \text{ hours} \approx 52 \text{ days}.$$

Even reducing this to 3 values per parameter yields $3^3 \times 5 \times 2 = 270$ hours. The practitioner cannot afford to evaluate all combinations exhaustively within a reasonable time. She needs a smarter strategy for *deciding which configurations to evaluate*.

This is the central question of hyperparameter search: given a fixed budget B (maximum number of model trainings), how should the budget be allocated across the hyperparameter space to maximize the chance of finding a good configuration?

1.2 Two Complementary Components

Every model selection workflow has two complementary components that always operate together:

- A **search strategy**: defines *which* configurations to evaluate. It proposes candidate configurations.
- An **evaluation method**: estimates how well each proposed configuration generalizes. It scores each candidate (using holdout, k-fold CV, nested CV, etc.).

These two components are independent: any search strategy can be combined with any evaluation method. In the examples from the Model Selection notes, grid search was the implicit search strategy paired with k-fold CV as the evaluation method. This section now makes the search strategy explicit and compares three approaches: **Grid Search**, **Random Search**, and **Bayesian Optimization**.

Hyperparameter Tuning

The process of systematically searching for a combination of hyperparameter values that maximizes model performance on unseen data. It requires choosing both a **search strategy** (which configurations to try) and an **evaluation method** (how to score each configuration).

Regardless of the strategy, the goal is always the same: find the configuration that yields the lowest **generalization error**. How to estimate that error reliably is the central question of model selection—covered in the companion notes. Here, the focus is on how to decide *which* configurations to feed into the evaluation loop.

Entry Ticket 1

Before reading further, answer from your knowledge of the Model Selection notes:

1. In the SVM tuning example (Section on k-fold CV), how were the hyperparameter values C and γ chosen for evaluation? Was that a deliberate strategy or an arbitrary listing?
2. What is the key difference between using the test set and the validation set during hyperparameter tuning?
3. If a model has 4 hyperparameters, each with 5 candidate values, and 10-fold CV is used to evaluate each combination, how many model trainings are required in total?

2 Grid Search

Grid search defines a finite set of candidate values for each hyperparameter and trains the model on *every* combination in the resulting Cartesian product. It is the simplest and most exhaustive search strategy.

2.1 Procedure

Given hyperparameters $h_1, h_2, \ldots, h_k$ with candidate sets $V_1, V_2, \ldots, V_k$, the total number of model trainings is:

$$N_{\text{grid}} = |V_1| \times |V_2| \times \cdots \times |V_k|.$$

For each configuration $(v_1, v_2, \ldots, v_k) \in V_1 \times \cdots \times V_k$, the model is trained and evaluated (via the chosen evaluation method). The configuration that achieves the lowest validation error is selected.

Consider tuning two hyperparameters: `max_depth` $\in \{2, 4, 6, 8, 10\}$ and regularization strength $\lambda \in \{0.001, 0.01, 0.1, 1.0\}$. This requires $5 \times 4 = 20$ model trainings. Adding a third hyperparameter with five candidate values raises the cost to $5 \times 4 \times 5 = 100$. In general, if each of the k hyperparameters has v candidate values, the cost is v^k: **exponential** in the number of hyperparameters, a phenomenon often called the *curse of dimensionality of search*.

Figure 1 illustrates the grid search procedure for two hyperparameters. Each blue dot represents one model training; every intersection of the grid is evaluated. The green circle marks the configuration with the lowest validation error.

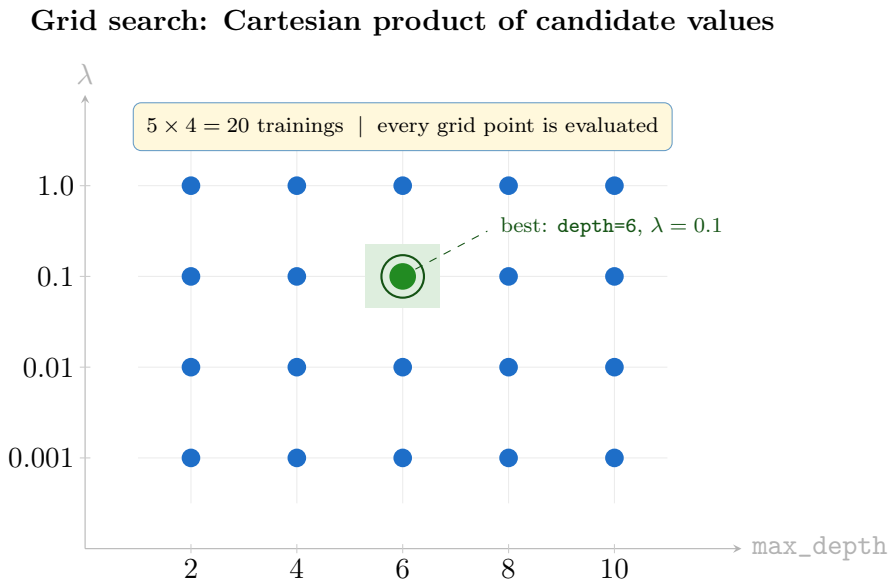

Figure 1: Grid search with two hyperparameters: `max_depth` $\in \{2, 4, 6, 8, 10\}$ and $\lambda \in \{0.001, 0.01, 0.1, 1.0\}$. Each blue dot represents one complete model training; every intersection of the grid is evaluated, for a total of $5 \times 4 = 20$ trainings. The green circle marks the best configuration found. Adding a third hyperparameter with 5 values would raise the total to 100.

2.2 Worked Example: Grid Search with 5-Fold CV

Consider the following scenario. A practitioner wants to tune an SVM with an RBF kernel on a development set of 500 examples. The hyperparameter grid is:

$$C \in \{0.1,\ 1,\ 10\} \qquad \gamma \in \{0.01,\ 0.1\}$$

She uses 5-fold cross-validation with accuracy as the metric. The mean CV accuracy (E_{CV}) and its standard deviation across folds for each of the $3 \times 2 = 6$ configurations are:

Config.	C	γ	E_1	E_2	E_3	E_4	E_5	$E_{\text{CV}} \pm \text{std}$
A	0.1	0.01	.71	.69	.72	.70	.71	0.706 ± 0.010
B	0.1	0.10	.74	.73	.75	.72	.74	0.736 ± 0.011
C	1	0.01	.80	.82	.79	.81	.80	0.804 ± 0.011
D	1	0.10	.85	.86	.84	.87	.85	$\mathbf{0.854 \pm 0.011}$
E	10	0.01	.83	.82	.84	.81	.83	0.826 ± 0.011
F	10	0.10	.84	.86	.85	.87	.83	0.850 ± 0.016

Questions.

1. How many total model trainings did this grid search require?
2. Which configuration should be selected, and why?
3. Config. F has the same E_{CV} (to three decimal places) as config. D but a higher standard deviation. Which would you prefer? Why?

4. After selection, what is the correct next step before touching the test set?

Solution.

(Q1) Total model trainings.

$$\underbrace{3}_{|C|} \times \underbrace{2}_{|\gamma|} \times \underbrace{5}_{k\text{-folds}} = 30 \text{ trainings.}$$

Each of the 6 configurations is trained and evaluated 5 times (once per fold).

(Q2) Configuration selection. Config. D ($C = 1$, $\gamma = 0.1$) achieves the highest $E_{\text{CV}} = 0.854$. It is selected as the best configuration (see Figure 2). The test set has not been consulted at any point in this process and remains unbiased.

(Q3) D vs. F: equal mean, different variance.

Both D and F achieve $E_{\text{CV}} \approx 0.850$–$0.854$, but their standard deviations differ:

Config.	C	γ	E_{CV}	std
D	1	0.10	0.854	0.011
F	10	0.10	0.850	0.016

Config. D is preferable for three reasons:

- **Higher mean accuracy** (0.854 vs. 0.850).
- **Lower standard deviation** (0.011 vs. 0.016): the performance of D is more consistent across folds, suggesting it is less sensitive to which particular examples end up in the training set. Config. F's higher std suggests it may be overfitting some folds while underfitting others, a sign of instability.
- **Lower** C: when two configurations have similar accuracy, the one with smaller C (stronger regularization) is generally preferred as it is less prone to overfitting on the final deployment data.

(Q4) Correct next step.

1. Retrain the SVM with $C = 1$, $\gamma = 0.1$ on the *entire development set* (all 500 examples). This produces the final model with more training data than any individual fold model had.
2. Only after this retraining, evaluate the final model on the held-out test set exactly once, and report the result.

Key takeaway. When two configurations have similar E_{CV}, prefer the one with lower standard deviation and lower complexity. The test set is never consulted during grid search; it waits until after retraining on the full development set.

Entry Ticket 2

A model is tuned using 5-fold CV over a grid of 20 hyperparameter combinations, and the best combination is selected.

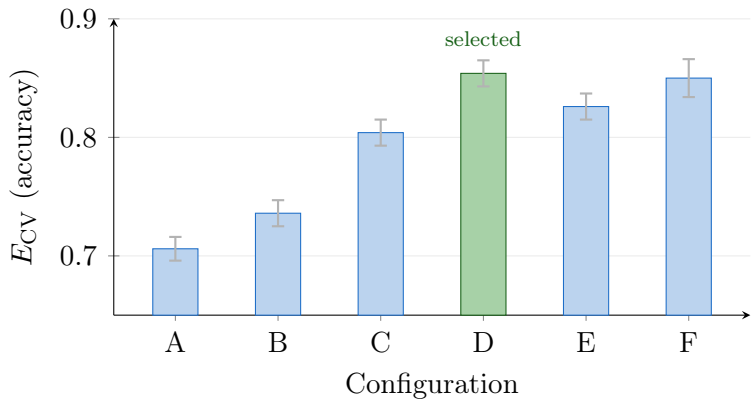

Figure 2: Mean 5-fold CV accuracy (E_{CV}) with error bars ($\pm$ one standard deviation across folds) for the six SVM configurations. Configuration D achieves the highest mean cross-validation accuracy and is therefore selected.

1. How many model trainings are required in total?
2. Why might the CV score of the *selected* configuration still be optimistically biased?
3. What additional step is needed before reporting final performance?
4. Suppose a second practitioner evaluates the same 20 combinations but uses 10-fold CV instead of 5-fold CV. How many model trainings does she need? Is her estimate expected to be more or less stable? Why?

2.3 Advantages

- Fully reproducible: the exact same configurations are always evaluated.
- Guarantees that every point on the predefined grid is assessed.
- Trivially parallelizable: each configuration is independent of the others.

2.4 Limitations

- Cost grows **exponentially** with the number of hyperparameters.
- The grid is discrete: if the optimum lies between two grid values, it is never found.
- If a hyperparameter is *irrelevant* to performance, the grid wastes evaluations by repeating equivalent experiments along that dimension. With k hyperparameters of which only one matters, a fraction $(1 - 1/|V_i|^{k-1})$ of all evaluations are redundant.

2.5 When to Use

Grid search is appropriate when the number of hyperparameters is small (1–2) and the candidate sets are well understood in advance. For larger spaces, the methods described below are more cost-efficient.

3 Random Search

Random search draws N configurations uniformly (or from a user-specified distribution) over the hyperparameter space and evaluates each one independently. Unlike grid search, *continuous* hyperparameters need not be discretized in advance: a range such as $\lambda \sim \text{LogUniform}(10^{-4}, 10^{0})$ is sampled directly, giving arbitrarily fine coverage of that axis.

3.1 Procedure

Each configuration is drawn independently from user-specified distributions over the hyperparameter space—uniform or log-uniform for continuous parameters, and uniform over a finite set for discrete ones. With a budget of B evaluations, exactly B configurations are sampled and evaluated. No information from previous evaluations influences subsequent draws.

The key theoretical property of random search is its **marginal coverage**: the number of distinct values tested along a single dimension h_i, ignoring all other dimensions. With k hyperparameters of which only $r \leq k$ affect performance:

Bergstra and Bengio [2012]

Grid: marginal coverage of $h_i = B^{1/k}$ Random: marginal coverage of $h_i = B$.

Random search is equivalent to performing a dense search on the relevant dimensions alone, allocating the full budget to the dimensions that matter.

Consider two hyperparameters: h_1 (important) and h_2 (less important). With a grid of m values per hyperparameter, the total budget is $B = m^2$. Grid search tests exactly m distinct values of h_1; each value of h_1 is paired with every candidate value of h_2, spreading the budget uniformly across both dimensions regardless of their relative importance. Random search with the same budget B draws each configuration independently; projected onto h_1 alone, this yields up to B distinct values, covering the important dimension far more densely.

Figure 3 illustrates this contrast using the same two hyperparameters introduced in the grid search example: λ (the regularization strength, a continuous parameter that strongly influences generalization) and `max_depth` (a discrete structural parameter that, for a fixed dataset, may contribute less to performance differences once it exceeds a moderate value). With a budget of 20 evaluations, grid search tests only 4 distinct values of λ, each paired with every candidate depth; random search draws 20 independent configurations and covers the full log-scale range of λ with near-complete density.

3.2 Worked Example: Marginal Coverage in Practice

Consider a practitioner tuning a regularized regression model with two hyperparameters:

- $\lambda \in [10^{-3}, 10^{0}]$ (regularization strength, *strongly* influences generalization error).
- $d \in \{2, 4, 6, 8, 10\}$ (polynomial feature degree, *weakly* relevant for this particular dataset once $d \geq 4$).

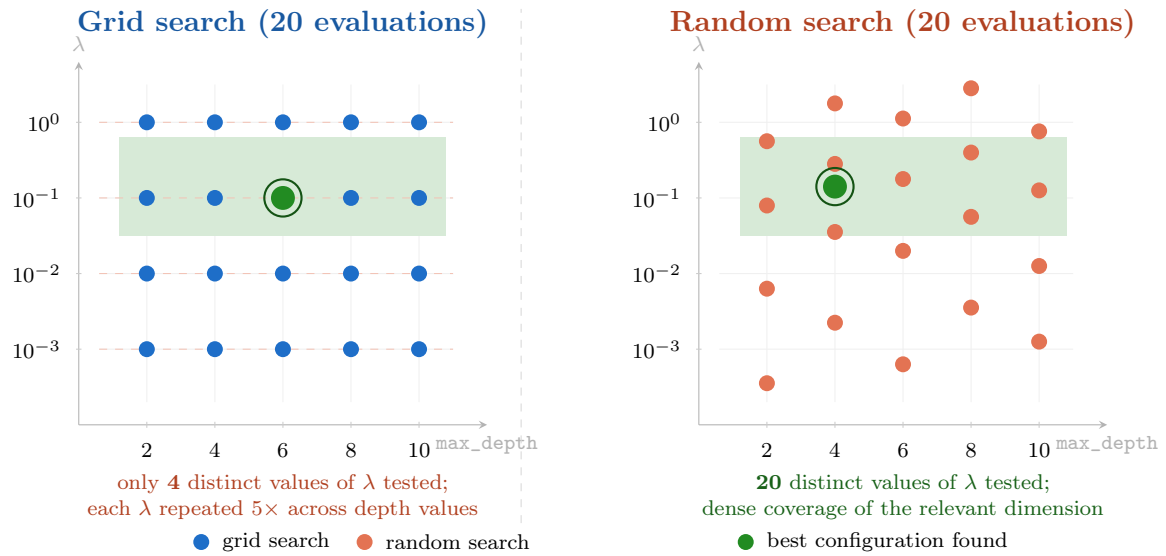

Figure 3: Grid search vs. random search with 20 evaluations each. Both panels share the same axes: `max_depth` $\in \{2, 4, 6, 8, 10\}$ (discrete, x-axis) and $\lambda \in [10^{-3}, 10^{0}]$ (continuous, y-axis). Left: the grid tests only 4 distinct values of λ; each value is repeated across all 5 depth values, wasting budget if depth has little effect on performance. Right: random search draws λ continuously from the full log-scale range and samples depth values without a fixed pattern, covering the relevant dimension (λ) with 20 distinct values rather than 4. The performance region is highlighted in green in both panels.

The total budget is $B = 25$ model trainings.

Grid search allocation. A natural 5×5 grid places the 25 evaluations at all combinations of $\lambda \in \{10^{-3}, 10^{-2.25}, 10^{-1.5}, 10^{-0.75}, 10^{0}\}$ and $d \in \{2, 4, 6, 8, 10\}$. The grid tests exactly $B^{1/k} = 25^{1/2} = 5$ distinct values of λ, each paired with every degree value.

Random search allocation. Random search with the same budget $B = 25$ draws λ from $\text{LogUniform}(10^{-3}, 10^{0})$ and d uniformly from $\{2, 4, 6, 8, 10\}$. Projected onto λ alone, this yields 25 distinct values—five times the coverage of grid search along the dimension that actually matters.

The table below shows a representative outcome. Validation MSE is simulated assuming the true optimum lies near $\lambda \approx 0.03$ (between the 3rd and 4th grid values) and that $d \geq 4$ makes little practical difference.

Grid search ($5 \times 5 = 25$ configs)				**Random search** ($B = 25$ configs)			
λ	d	Val. MSE		λ	d	Val. MSE	
$10^{-3.00}$	2	1.41		$10^{-2.82}$	4	1.28	
$10^{-3.00}$	4	1.39		$10^{-2.51}$	8	1.22	
$10^{-3.00}$	6	1.40		$10^{-2.10}$	6	1.05	
$10^{-3.00}$	8	1.38		$10^{-1.88}$	4	0.94	
$10^{-3.00}$	10	1.39		$10^{-1.73}$	10	0.88	
$10^{-2.25}$	2	1.18		$10^{-1.62}$	2	0.81	
$10^{-2.25}$	4	1.15		$10^{-1.55}$	6	0.76	
$10^{-2.25}$	6	1.16		$10^{-1.48}$	8	0.71	
$10^{-2.25}$	8	1.14		$10^{-1.40}$	4	0.65	
$10^{-2.25}$	10	1.15		$10^{-1.31}$	10	0.62	
$10^{-1.50}$	2	0.87		$10^{-1.22}$	6	0.59	
$10^{-1.50}$	4	0.84		$10^{-1.12}$	2	0.56	
$10^{-1.50}$	6	0.85		$10^{-1.05}$	8	**0.53**	$\leftarrow$ best
$10^{-1.50}$	8	0.83		$10^{-0.96}$	4	0.55	
$10^{-1.50}$	10	0.84		$10^{-0.88}$	10	0.57	
$10^{-0.75}$	2	0.74		$10^{-0.79}$	6	0.61	
$10^{-0.75}$	4	**0.71**	$\leftarrow$ best	$10^{-0.72}$	2	0.65	
$10^{-0.75}$	6	0.72		$10^{-0.63}$	8	0.70	
$10^{-0.75}$	8	0.70		$10^{-0.54}$	4	0.76	
$10^{-0.75}$	10	0.72		$10^{-0.42}$	6	0.84	
$10^{0.00}$	2	0.91		$10^{-0.33}$	10	0.91	
$10^{0.00}$	4	0.89		$10^{-0.22}$	2	0.97	
$10^{0.00}$	6	0.90		$10^{-0.14}$	8	1.05	
$10^{0.00}$	8	0.88		$10^{-0.07}$	4	1.08	
$10^{0.00}$	10	0.90		$10^{-0.02}$	6	1.11	

Questions.

1. What is the best validation MSE found by each strategy, and which value of λ does it correspond to?
2. The true optimum is near $\lambda \approx 0.03$ ($= 10^{-1.52}$). Neither strategy hits it exactly. Which strategy gets closer, and why?
3. How many of the 25 grid evaluations could have been eliminated without changing the best-found configuration? What does this suggest about wasted budget?
4. Suppose a third hyperparameter $\alpha \in [0, 1]$ is added (also weakly relevant). Recompute the marginal coverage of λ for (a) a $5 \times 5 \times 5$ grid search and (b) random search, both with budget $B = 125$.

Solution.

(Q1) Best configuration found.

Strategy	**Best MSE**	λ	d
Grid search	0.71	$10^{-0.75} \approx 0.178$	4
Random search	0.53	$10^{-1.05} \approx 0.089$	8

Random search found a configuration with MSE 0.53 vs. 0.71 for grid search—a meaningful improvement with the same budget.

(Q2) Proximity to the true optimum. The true optimum is near $\lambda \approx 0.03 = 10^{-1.52}$. Grid search tested $\lambda = 10^{-1.50}$ as its closest value (the third grid point), finding MSE ≈ 0.83. Random search sampled $\lambda = 10^{-1.05}$, which is farther from $10^{-1.52}$ in log scale, yet yielded lower MSE (0.53) because the random draws cover the intermediate region $[10^{-1.6}, 10^{-0.9}]$ with multiple points—including $\lambda = 10^{-1.05}$, $10^{-1.12}$, $10^{-1.22}$—while grid search places all five evaluations at fixed log-spaced values that happen to straddle the optimum without landing in the best sub-region.

(Q3) Wasted budget in grid search. The 5 rows with $\lambda = 10^{-3.00}$ and the 5 rows with $\lambda = 10^{0.00}$ all produce MSE above 0.88. Removing these 10 configurations entirely does not change the winning configuration ($\lambda = 10^{-0.75}$, $d = 4$). Moreover, within any fixed value of λ, the variation across degree values is at most 0.04—meaning 5 repeated evaluations at the same λ recover nearly identical information. If d were truly irrelevant, 4 of every 5 evaluations at the same λ would be redundant. In this example, at least 10–15 of 25 evaluations contributed little new information.

(Q4) Marginal coverage with a third hyperparameter.

With $k = 3$ and budget $B = 5^3 = 125$:

$$\text{Grid: marginal coverage of } \lambda = B^{1/k} = 125^{1/3} = 5 \text{ distinct values.}$$

$$\text{Random: marginal coverage of } \lambda = B = 125 \text{ distinct values.}$$

Adding the third (irrelevant) hyperparameter *does not change* random search's coverage of λ at all: it still tests 125 distinct values of λ. Grid search is unchanged too in absolute terms (still 5 values), but the budget is now three times larger for the same marginal coverage. The relative efficiency of random search over grid search grows as more irrelevant dimensions are added.

Key takeaway. The advantage of random search is not that it is lucky—it is that it allocates the full budget to every dimension simultaneously. When some dimensions are uninformative, grid search wastes evaluations replicating the same values of irrelevant hyperparameters, while random search continues diversifying along all axes. The worked example makes this concrete: with the same 25 evaluations, random search found a configuration 25% better in MSE than grid search, simply by probing finer resolution along λ.

Entry Ticket 3

Reflect on the grid search vs. random search comparison:

1. A model has 5 hyperparameters, each sampled with $m = 4$ candidate values. Compute the marginal coverage for a relevant dimension under (a) grid search and (b) random search with the same budget $B = 4^5$.
2. If you know in advance that 3 of the 5 hyperparameters are irrelevant, which strategy do you prefer and why?
3. Give an example of a hyperparameter for which using a LogUniform distribution instead of a uniform distribution is more appropriate. Justify your choice.

3.3 Advantages

- Tests more distinct values per relevant dimension for the same budget.
- Works naturally with continuous ranges and non-uniform distributions (e.g. $\lambda \sim$ LogUniform; learning rate $\sim$ Uniform$(10^{-4}, 10^{-1})$).
- Easily parallelizable; each draw is independent of the others.
- Empirically competitive with or superior to grid search in most settings.

3.4 Limitations

- Does not guarantee systematic coverage: with small budgets, an important region may happen to receive very few samples.
- Memoryless: each evaluation is independent of previous results; no directed exploration takes place.
- For very small budgets ($B \lesssim 10$), the variance of the best observed configuration can be large.

3.5 When to Use

Random search is the default first choice when the number of hyperparameters is moderate (3–6) and there is little prior knowledge about which values are promising. For very limited budgets or high-dimensional spaces, Bayesian Optimization is preferable.

4 Bayesian Optimization

Random search is memoryless: each configuration is drawn independently of all previous results, so no information is accumulated across evaluations. This is efficient when the budget is large relative to the hyperparameter search space Λ, but wasteful when each model training is expensive.

Bayesian optimization (BO) breaks this limitation by maintaining a probabilistic *surrogate model* of the objective function $f(\lambda) = \text{validation error}(\lambda)$. After each evaluation, the surrogate is updated to incorporate the new observation; an *acquisition function* then

selects the next configuration to evaluate by trading off two competing goals: **exploration** of regions where the surrogate is uncertain, and **exploitation** of regions already known to perform well.

4.1 Core Idea

The key distinction from grid and random search is that BO *learns* from previous evaluations. Instead of proposing configurations blindly, it builds an increasingly accurate model of the error landscape and uses that model to decide where to search next.

Two components make this possible:

- A **surrogate model** (typically a Gaussian process, GP) that, given all configurations evaluated so far, produces for any untested configuration λ both a predicted validation error $\mu(\lambda)$ and an uncertainty estimate $\sigma(\lambda)$. Regions near previously evaluated points have low uncertainty; unexplored regions have high uncertainty.
- An **acquisition function** $\alpha(\lambda)$ that combines $\mu(\lambda)$ and $\sigma(\lambda)$ into a single score. The next configuration to evaluate is $\lambda_{n+1} = \arg\max_\lambda \alpha(\lambda)$. A common choice is **Expected Improvement** (EI), which is large both where the surrogate predicts a good outcome *and* where uncertainty is high, automatically balancing exploration and exploitation.

Figure 4 illustrates two consecutive iterations of Bayesian optimization (BO) on a one-dimensional search space. In each panel, the current Gaussian Process (GP) surrogate model is shown as a solid curve representing the posterior mean, together with a shaded band indicating the 95% posterior confidence interval. The acquisition function—Expected Improvement (EI), rescaled vertically for visualization—is plotted below the horizontal axis.

In the first iteration (left panel), the GP surrogate is fitted to three previously evaluated configurations. Because large portions of the search space remain unexplored, the posterior uncertainty is relatively high, particularly in the right-hand region. EI balances two factors: the surrogate's predicted objective value (exploitation) and the model's uncertainty (exploration). Consequently, EI attains its maximum in that unexplored region, suggesting the next hyperparameter configuration to evaluate.

After this evaluation is performed, the new observation is incorporated into the dataset and the GP posterior is updated (right panel). As expected, the uncertainty band shrinks around the newly sampled location, reflecting the additional information obtained about the objective function. With the surrogate model now more confident in that region, the EI surface changes accordingly and its maximum shifts to another location that represents the most promising trade-off between high predicted performance and remaining uncertainty. This process then continues iteratively until the evaluation budget is exhausted or convergence is achieved.

4.2 Advantages

- **Sample-efficient:** concentrates evaluations in promising regions, typically finding good configurations in far fewer evaluations than grid or random search.
- **Handles continuous and mixed spaces naturally:** no discretization needed.

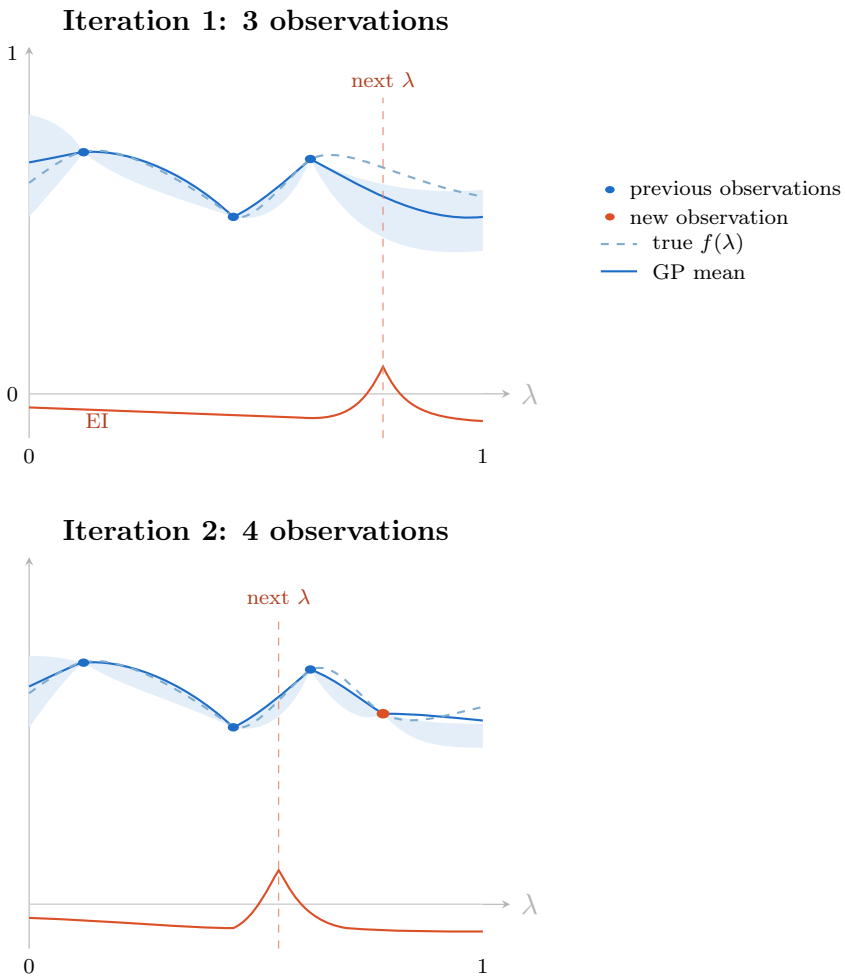

Figure 4: Two consecutive iterations of Bayesian optimization in a one-dimensional search space. The top panel shows the surrogate model and acquisition function after three observations. The bottom panel shows the updated model after evaluating the point selected by Expected Improvement.

- **Principled uncertainty quantification:** the surrogate drives the exploration–exploitation balance automatically.
- **Increasingly advantageous as per-evaluation cost grows:** when training a single model takes hours, saving even a few evaluations has significant practical value.

4.3 Limitations

- **Not natively parallelizable:** each evaluation must be completed before the surrogate is updated and the next point is selected.
- **Surrogate overhead grows with the number of observations:** fitting a Gaussian process scales as $\mathcal{O}(n^3)$.
- **Less effective in very high-dimensional spaces:** when $k \gg 20$, random search may remain competitive.

4.4 When to Use

Bayesian optimization is the method of choice when the per-evaluation cost is high (e.g. training a deep neural network or a large ensemble), the budget is limited ($B \lesssim 50$–100 evaluations), and the hyperparameter space is low-to-moderate dimensional ($k \lesssim 10$–20). For cheap models with large budgets, random search is often sufficient.

Entry Ticket 4

Bayesian optimization vs. the other strategies:

1. Explain in your own words why random search is "memoryless" while Bayesian optimization is not. What concrete advantage does this provide?
2. In which specific scenario would you prefer random search over Bayesian optimization, despite BO's theoretical advantage? Justify with a practical argument.
3. A team trains a large language model and can afford at most 20 hyperparameter evaluations. Each training run takes 8 hours. Which strategy would you recommend and why?

References

James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305, 2012.